

Hyperbolic Surface Codes

Hyosang Kang (DGIST)

Monash University

29 July 2026

Hyperbolic surface code: what, why, and how?

1. What is a (hyperbolic) surface code?

- Quantum error correction code
- Based on a (hyperbolic) lattice structure

2. Why is a (hyperbolic) surface code important?

- LDPC, CSS, Fault tolerant
- High threshold rate

3. How do we construct a hyperbolic surface code?

- Hyperbolic triangles

Quantum error correction code

1. Why do we need an error correction?

- Qubits have **high decoherence** rates: any quantum information is lost quickly.
- **No cloning theorem**: Quantum information cannot be copied.
- A quantum state **collapses** to its observed state after the measurement.

2. How do we correct a quantum error?

- A qubit $|\phi\rangle$ is a superposition of two base states $|0\rangle$ and $|1\rangle$.

$$|\phi\rangle = \alpha|0\rangle + \beta|1\rangle \in \mathcal{H}^1 \rightarrow \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \in M(2, 1; \mathbb{C})$$

- A quantum error on the qubit $|\phi\rangle$ reduces down to X (bit flip) and Z (phase flip) errors

$$X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle, \quad Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle$$

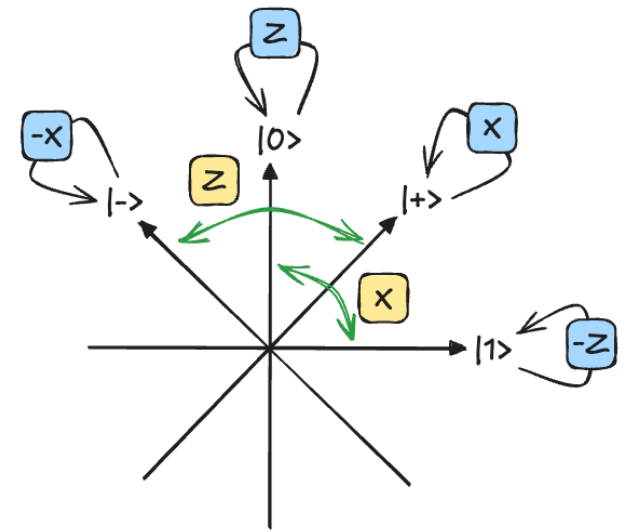
- A QEC can *detect* X or Z error(s) by the **syndrome measurement**, and the error correction is done retrospectively.

Measurement basis

- There are two (orthonormal) basis of $\mathcal{H}^1$: $\mathcal{B}_{01} = \{|0\rangle, |1\rangle\}$ and $\mathcal{B}_{+-} = \{|+\rangle, |-\rangle\}$.

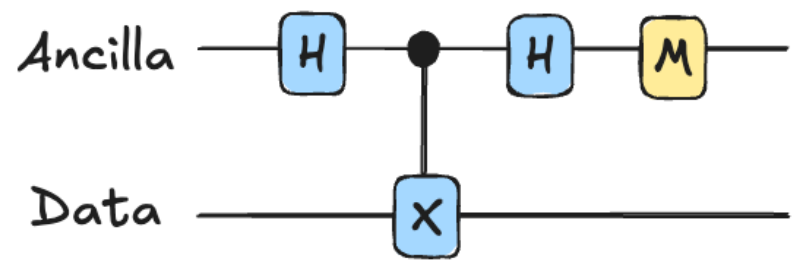
$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

- $|+\rangle, |-\rangle$ are the eigenvectors of X , and $|0\rangle, |1\rangle$ are the eigenvectors of Z .
- X flips $|0\rangle, |1\rangle$, and Z flips $|+\rangle, |-\rangle$.
 $X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle, \quad Z|+\rangle = |-\rangle, \quad Z|-\rangle = |+\rangle$

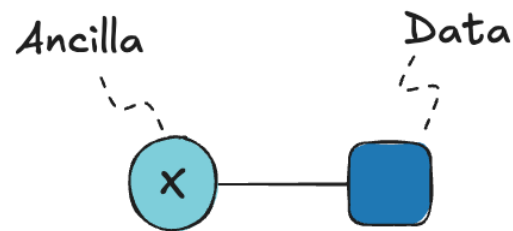


Syndrome measurement

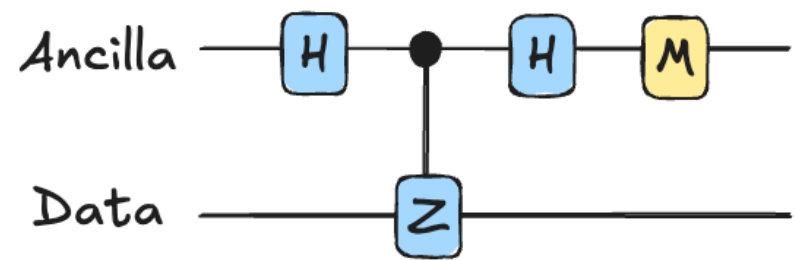
- X stabilizer



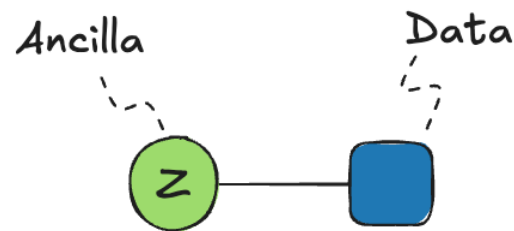
Detects Z error



- Z stabilizer

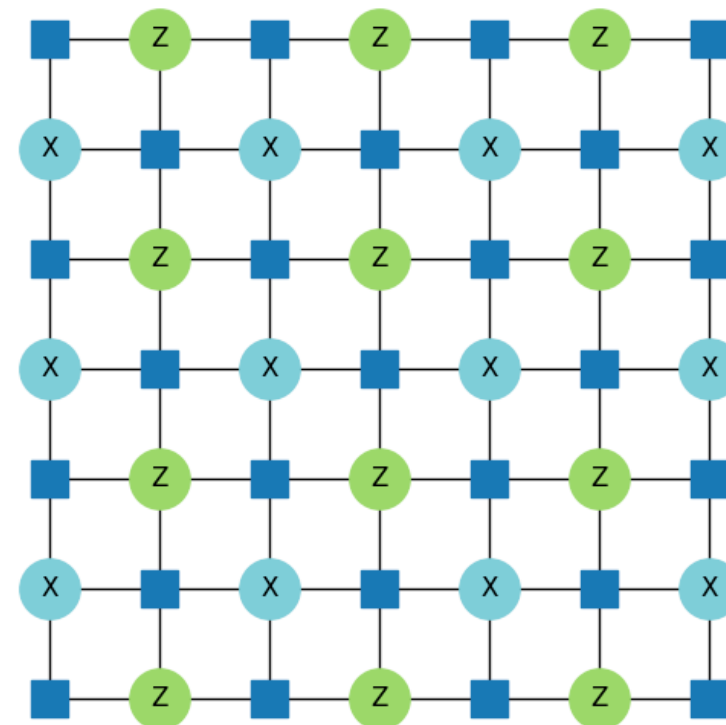


Detects X error



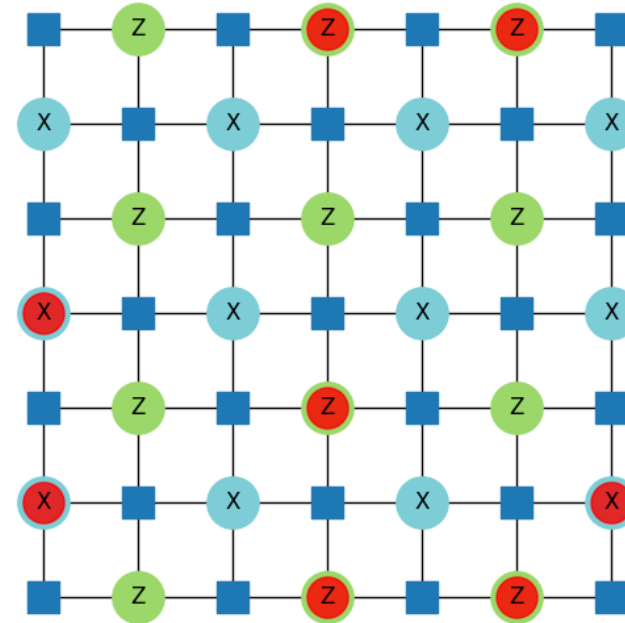
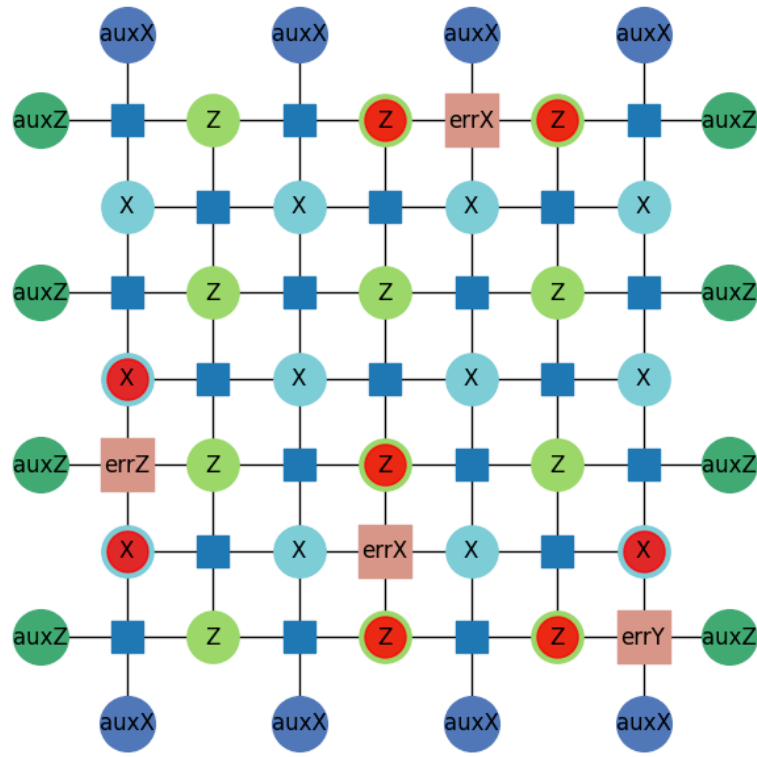
The surface code

- The size is $L = 6$ (must be even)
- The number of data qubits $n = \frac{L^2 + 2L + 2}{2}$.
- The number of stabilizer qubits $m = \frac{L^2 + 2L}{2}$.
- The number of **logical** qubit $k = n - m = 1$.

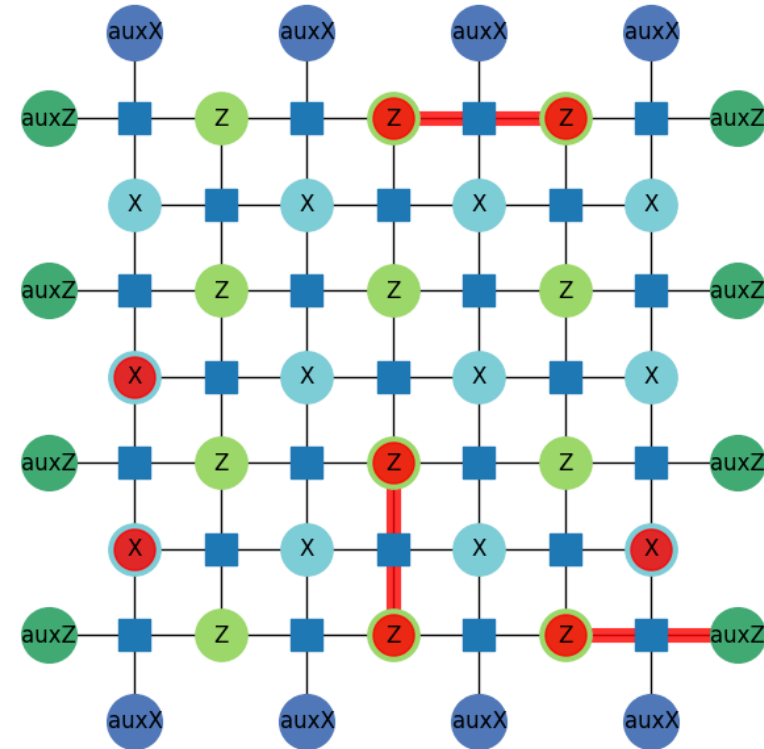
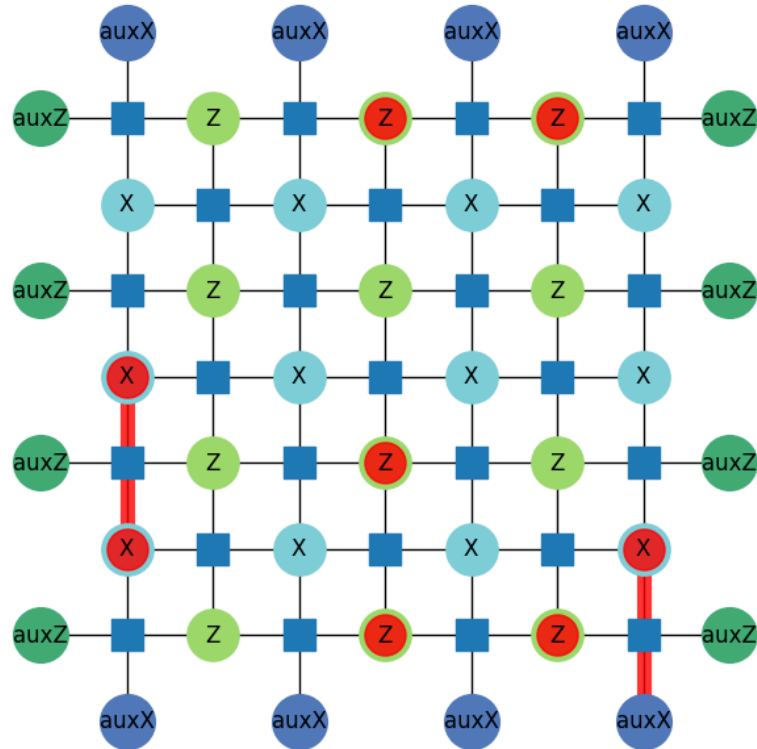


Error correction (success)

- Problem

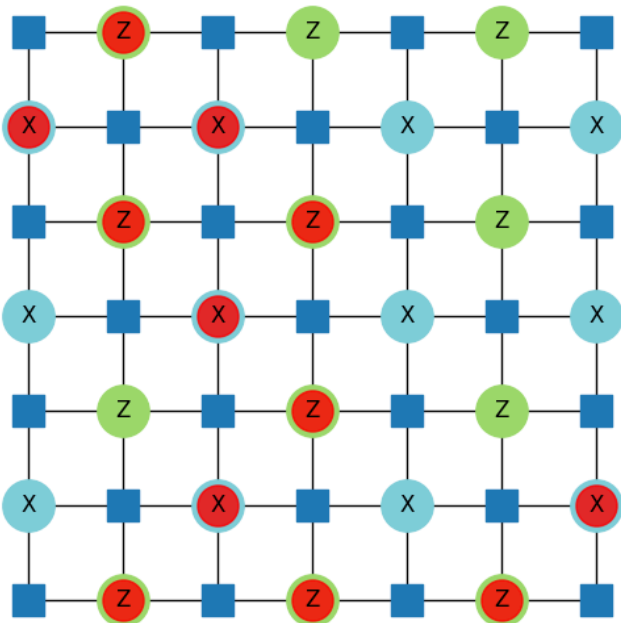
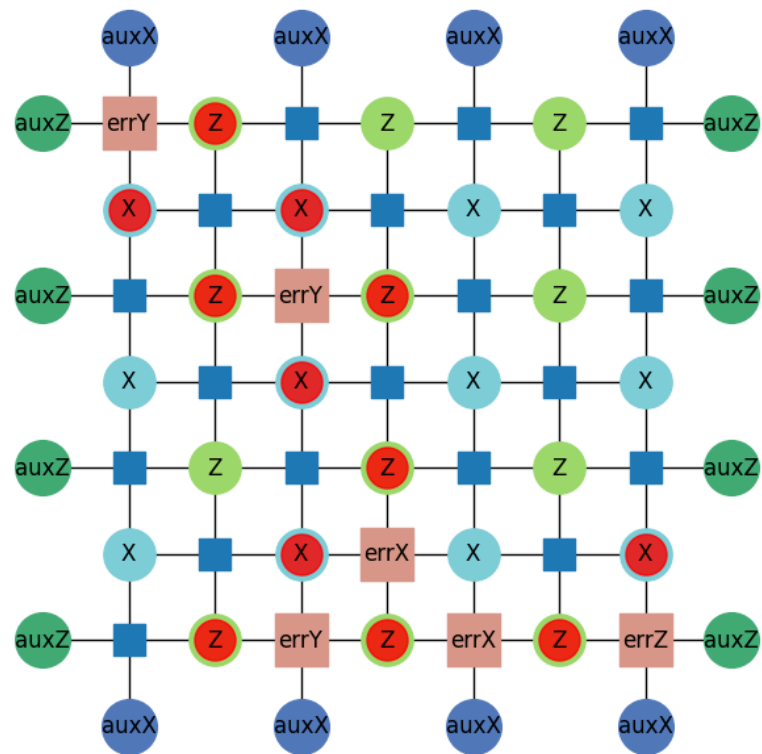


- **Solution (minimum weight perfect matching)**

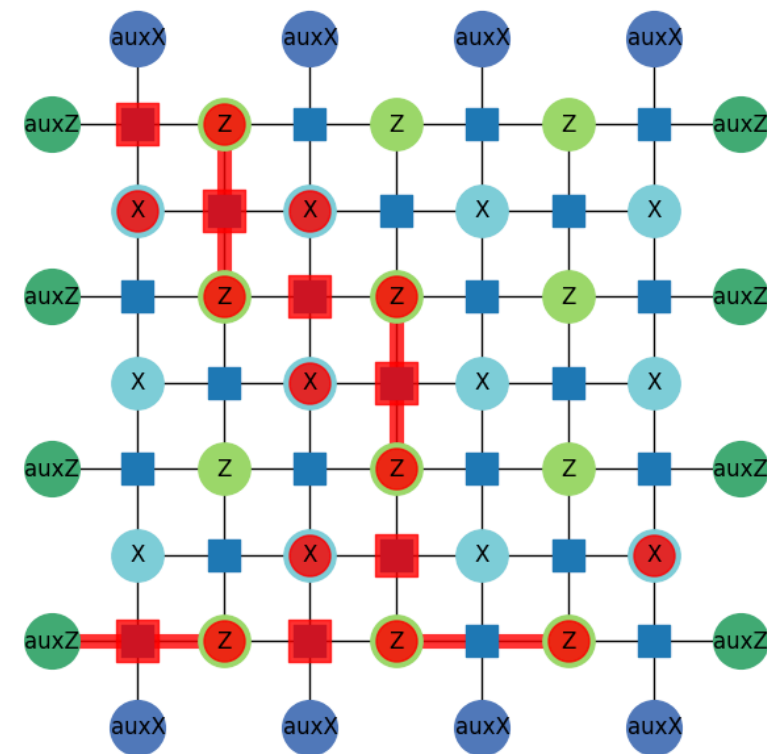
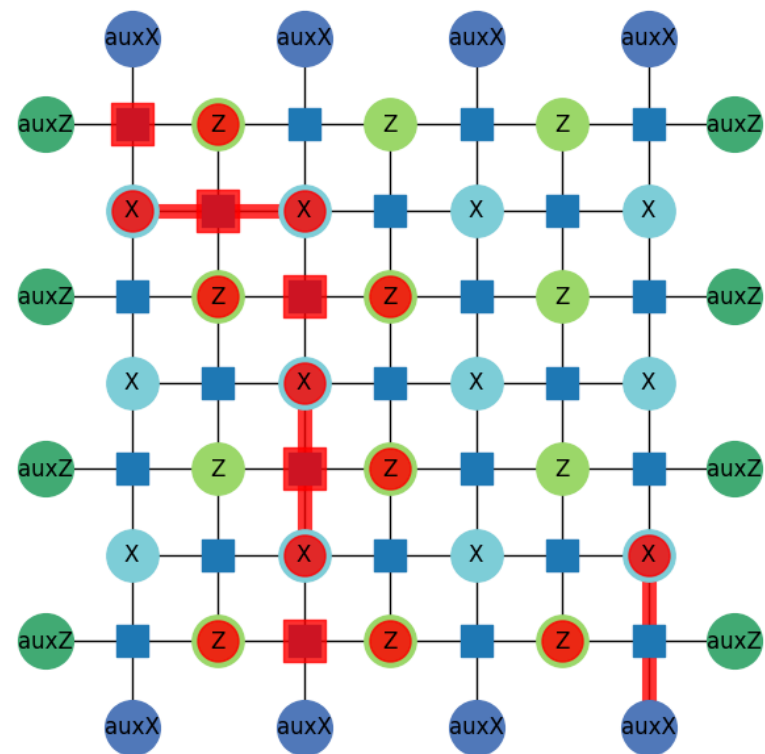


Error correction (failure)

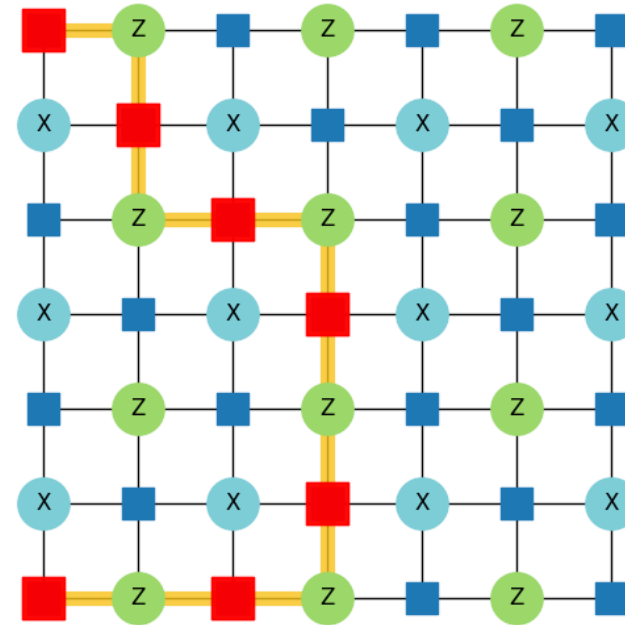
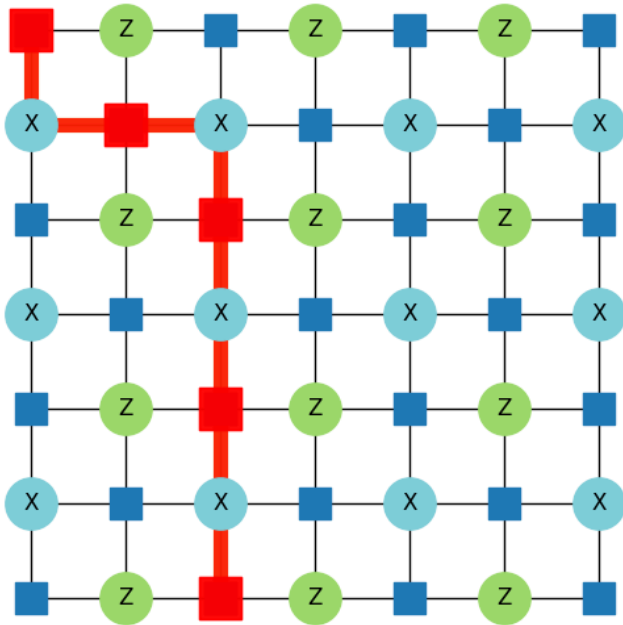
- Problem



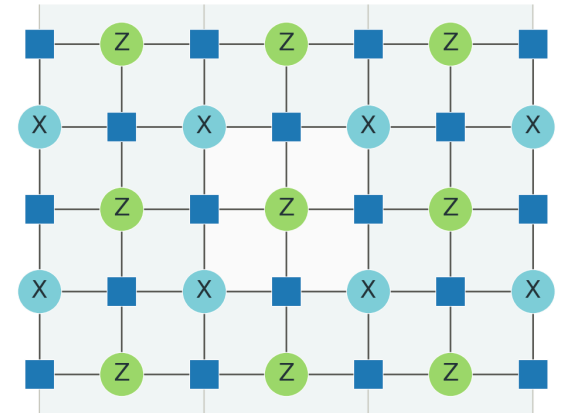
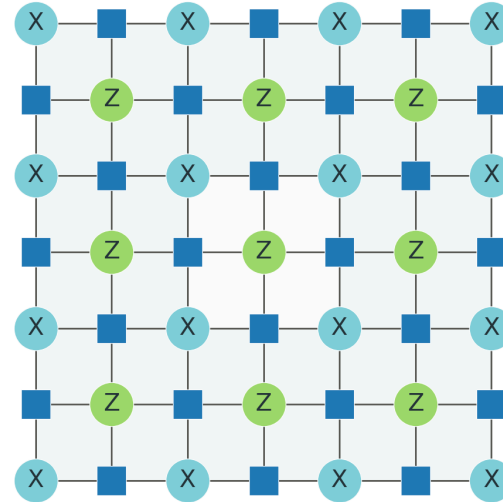
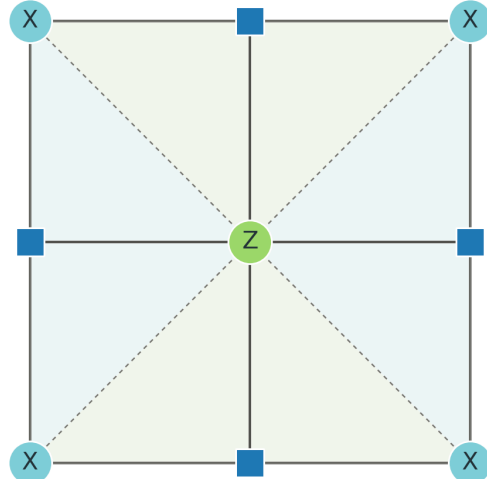
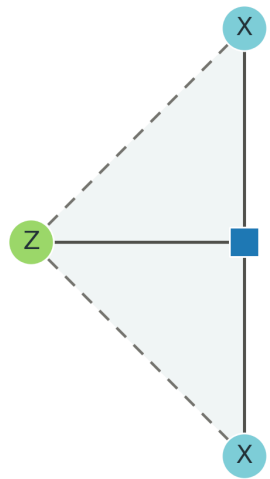
- Solution (mwpw)



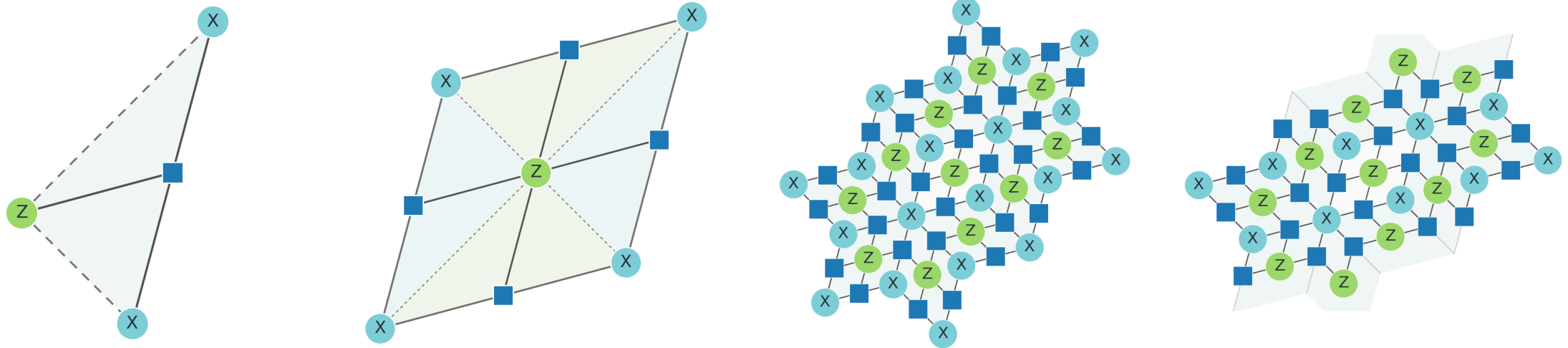
- Logical error



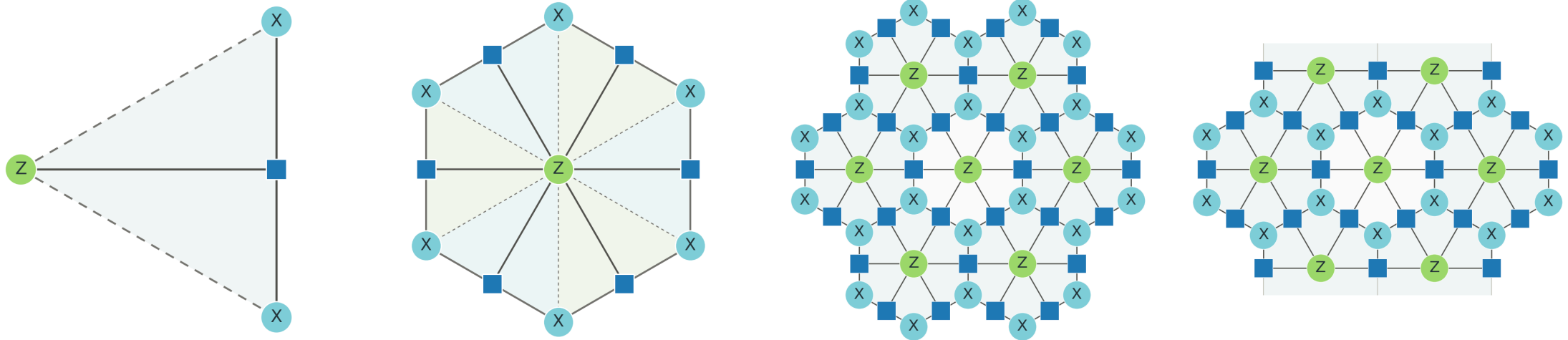
Construction of the $(2,4,4)$ surface code



Construction of $(2,3,6)$ -surface code

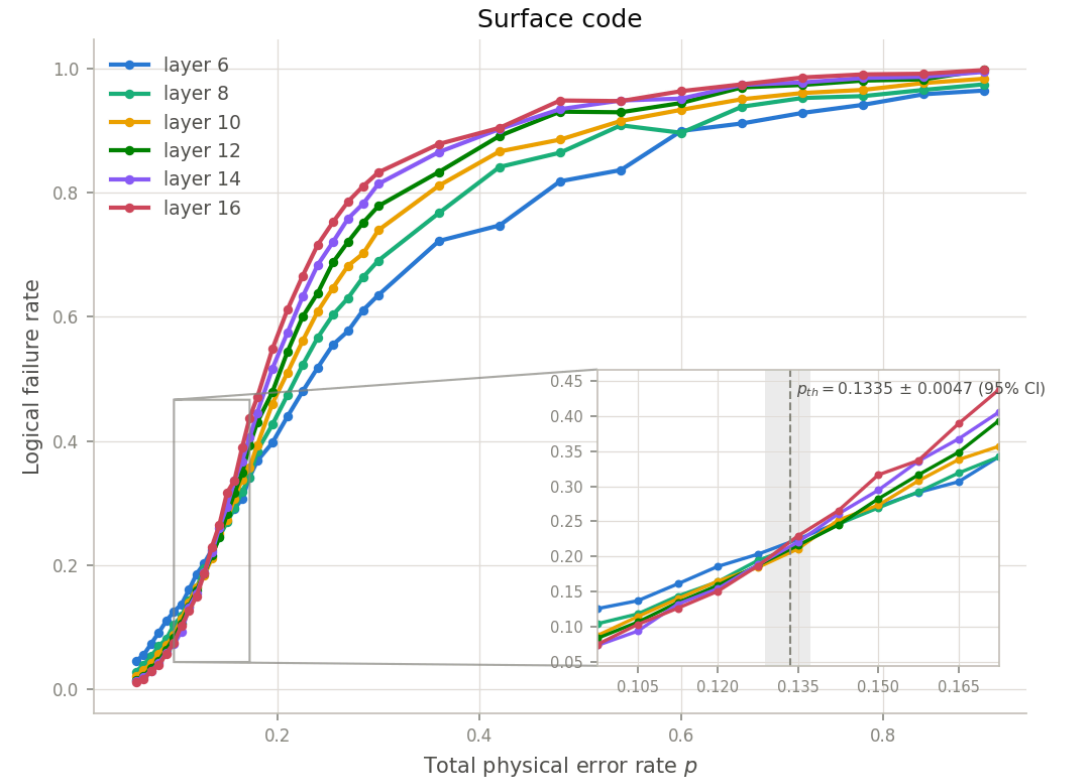


Construction of $(3,3,3)$ -surface code



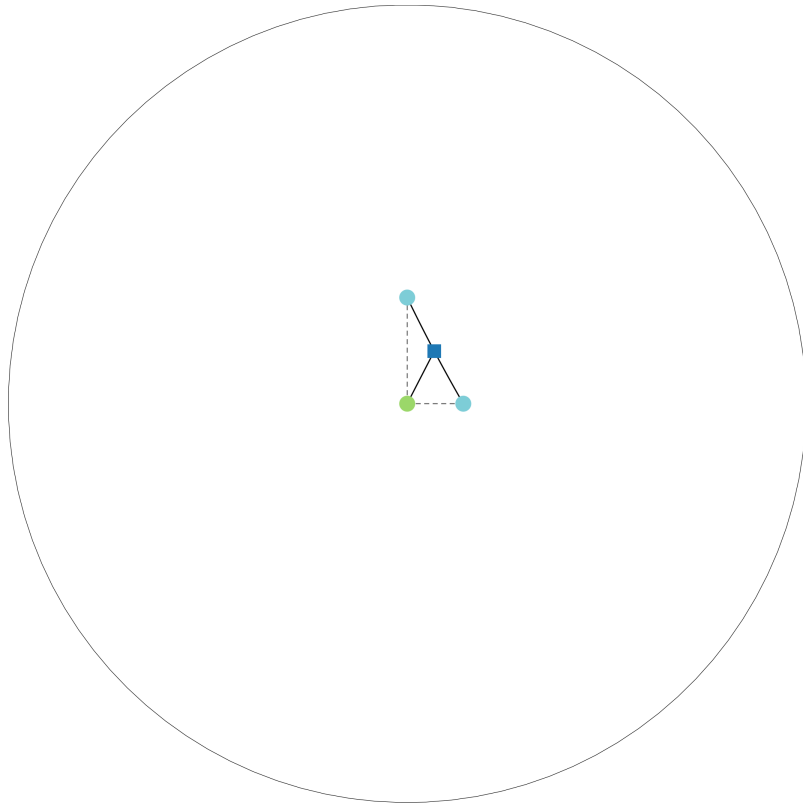
Error threshold

- At low physical error rate, logical error rate is lower than physical error rate
- Below **threshold**: larger codes reduce logical errors.
- Above **threshold**: larger codes do not help.

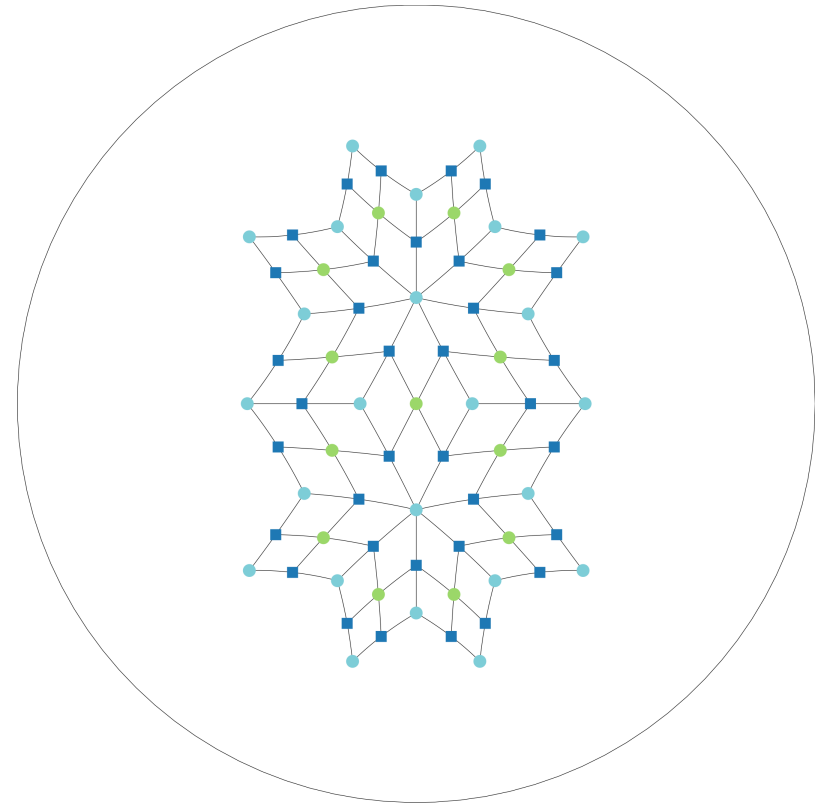


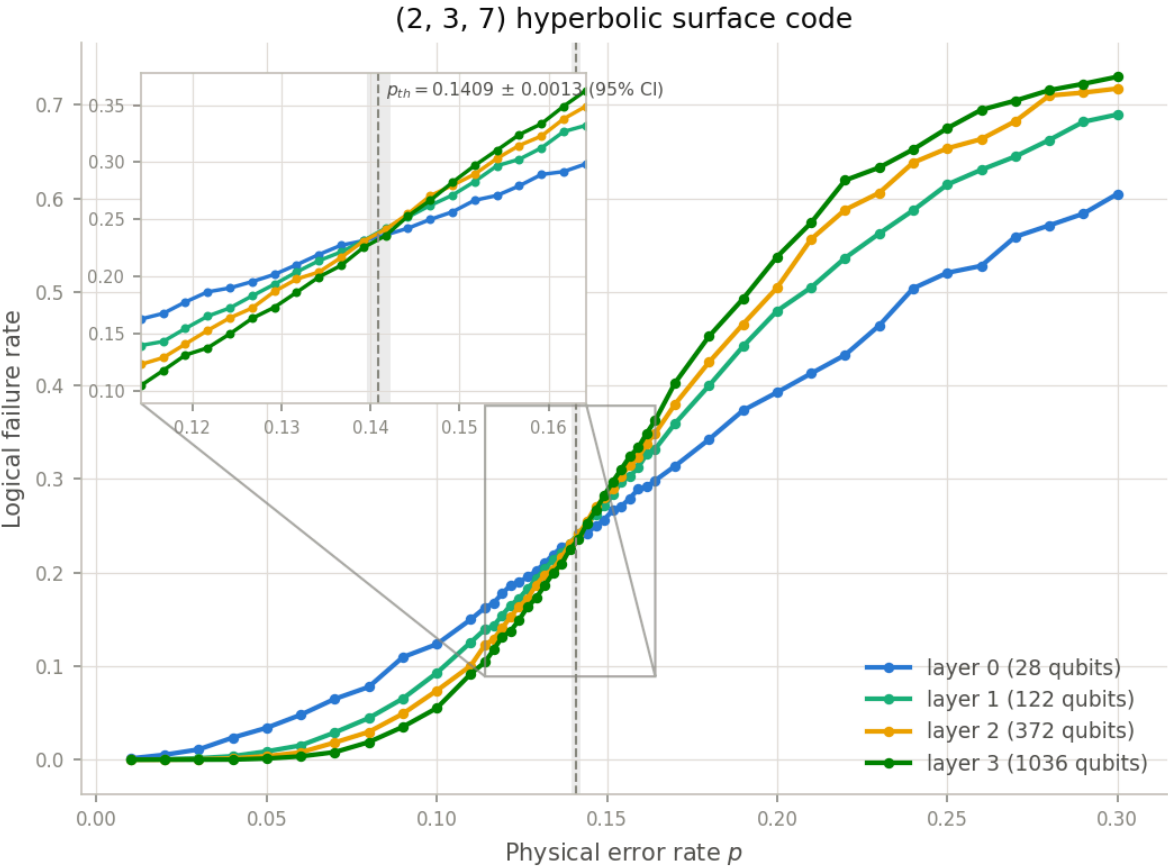
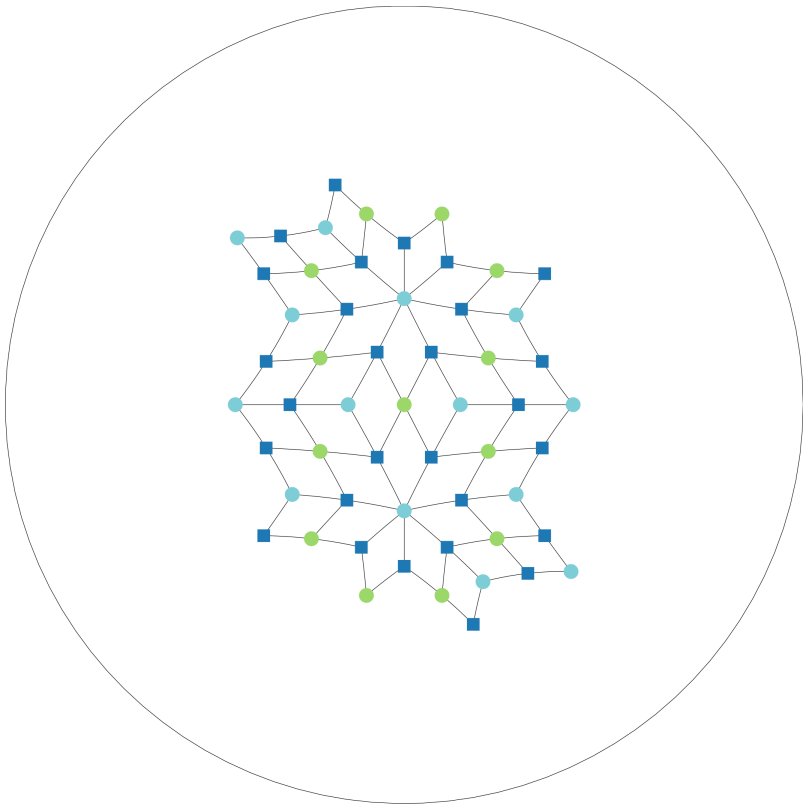
$(2, 3, 7)$ -hyperbolic surface code

Triangle tile



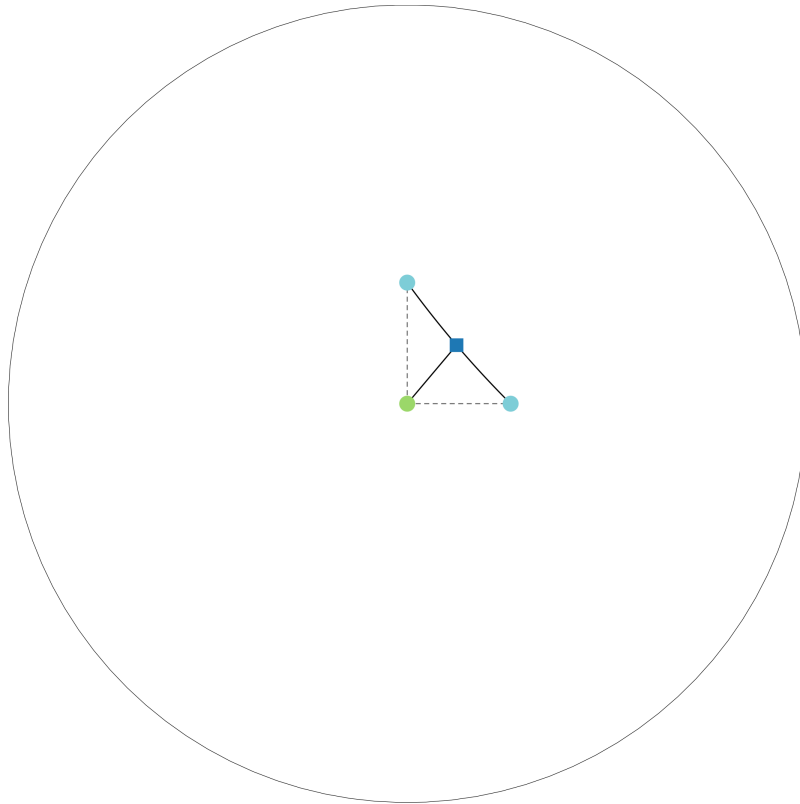
Layer 0 without trimming



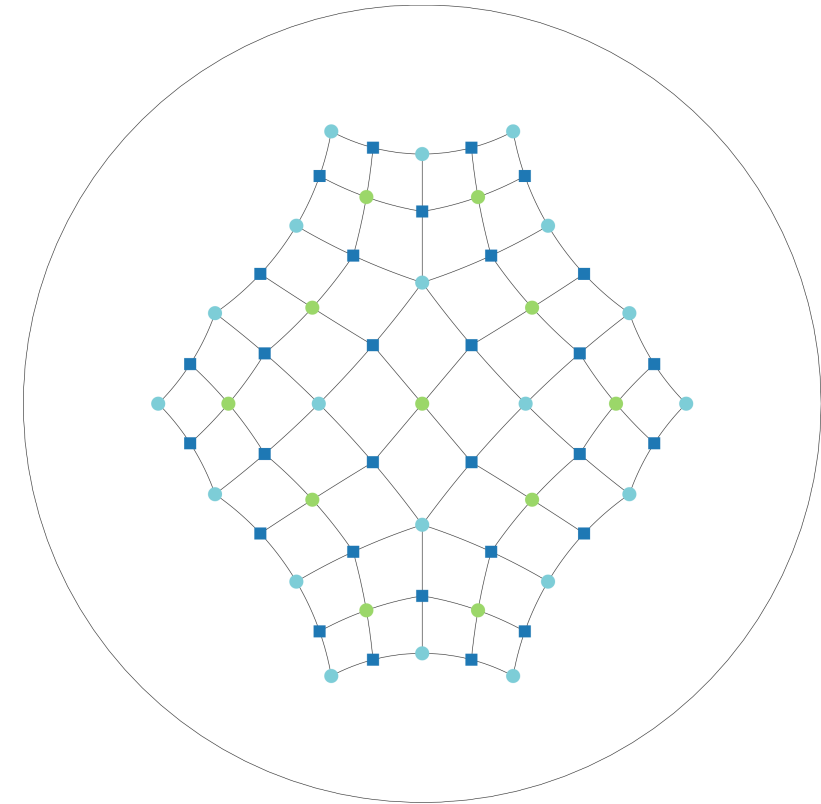


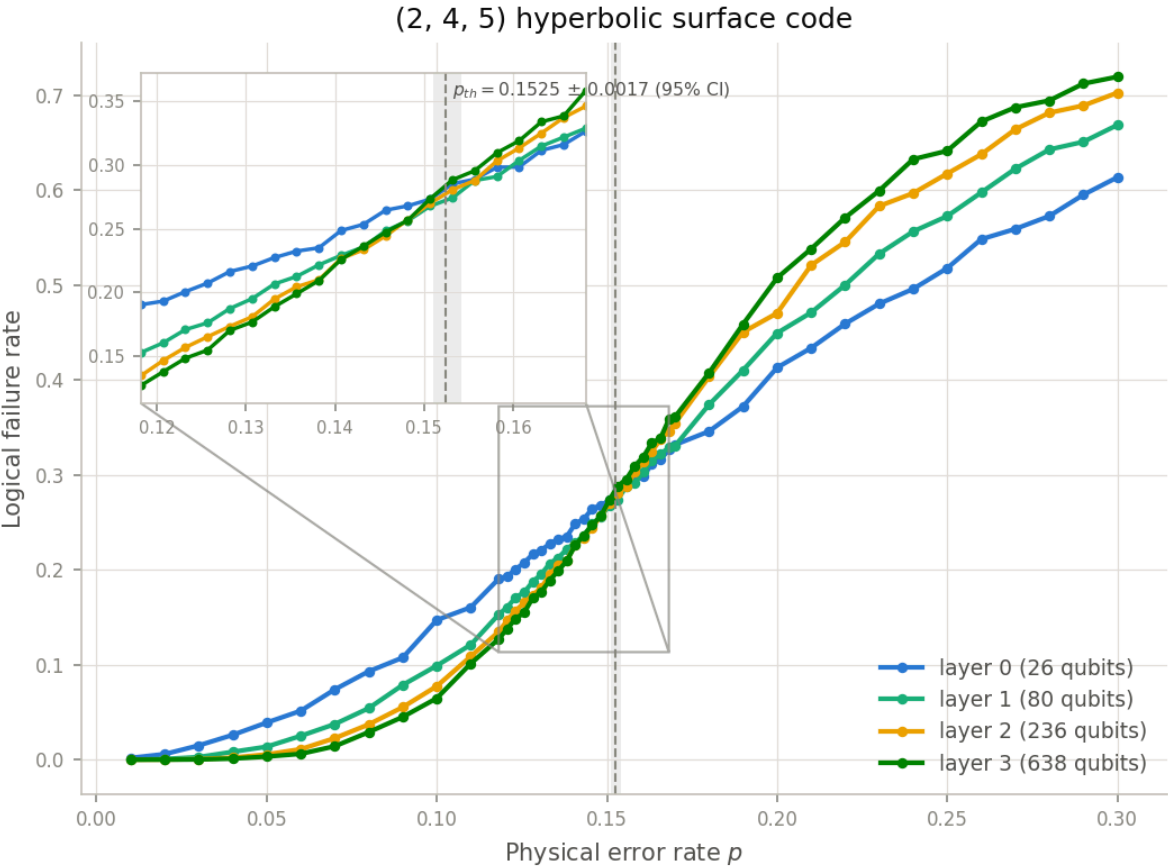
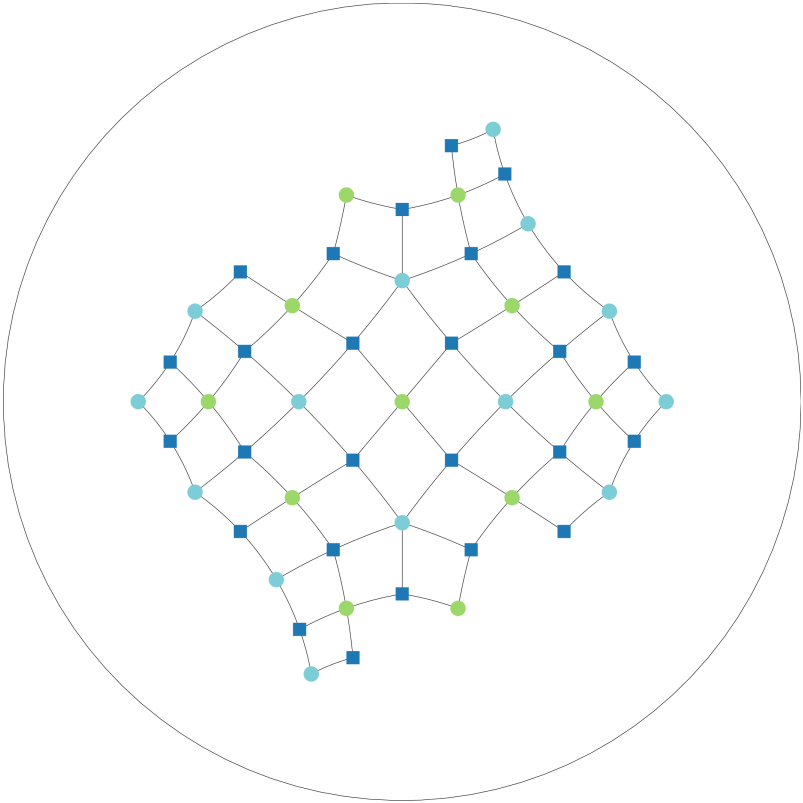
$(2, 4, 5)$ -hyperbolic surface code

Triangle tile



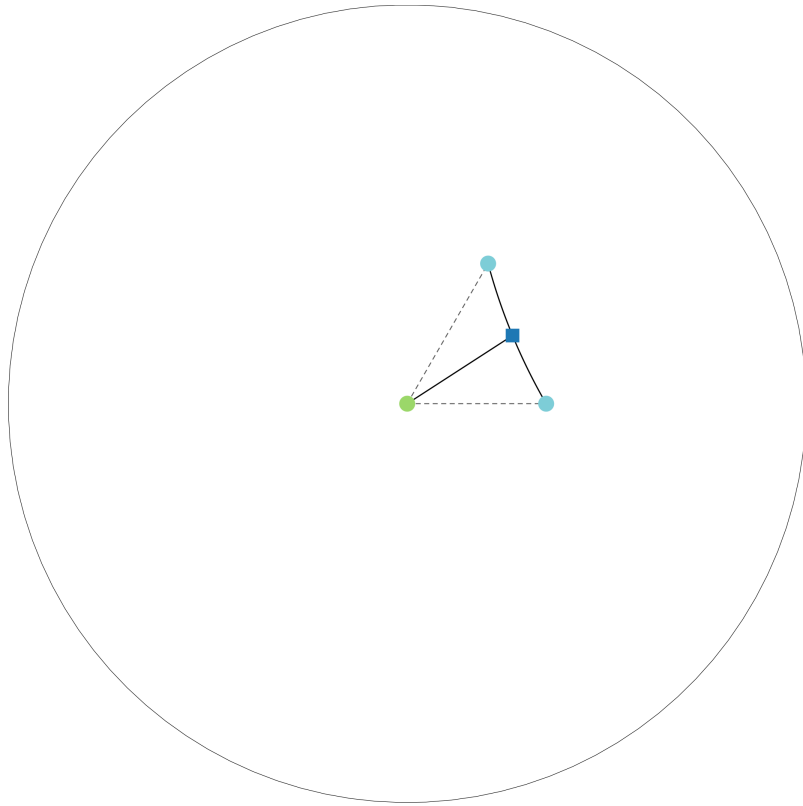
Layer 0 without trimming



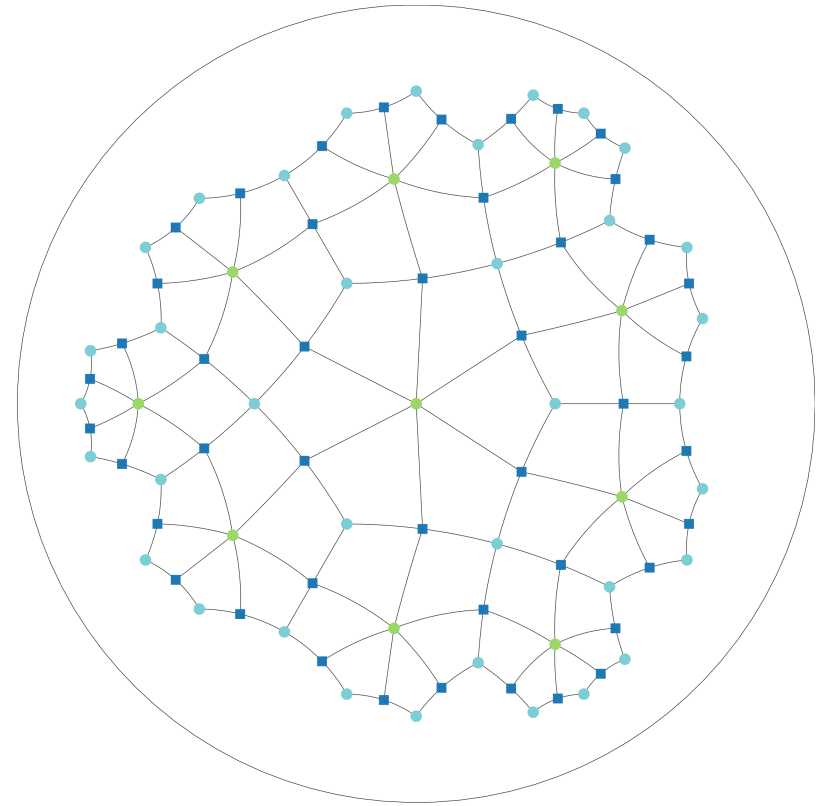


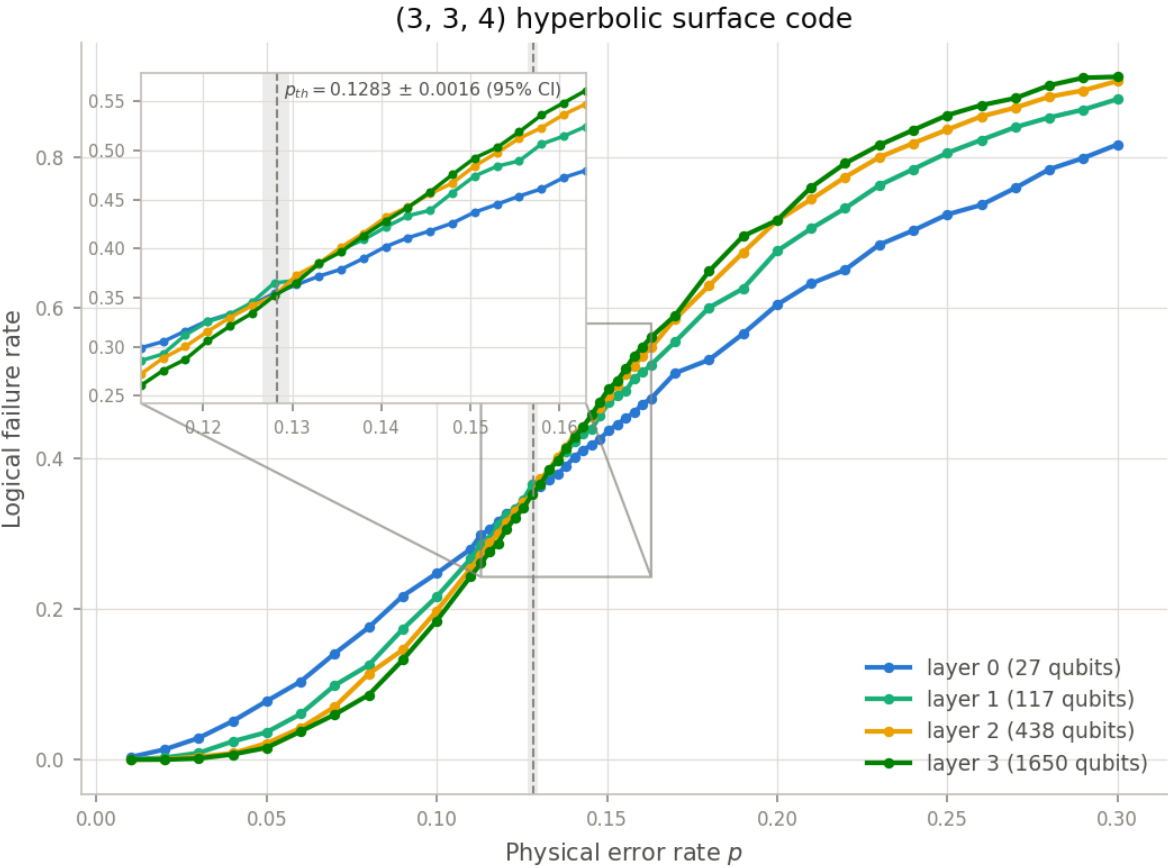
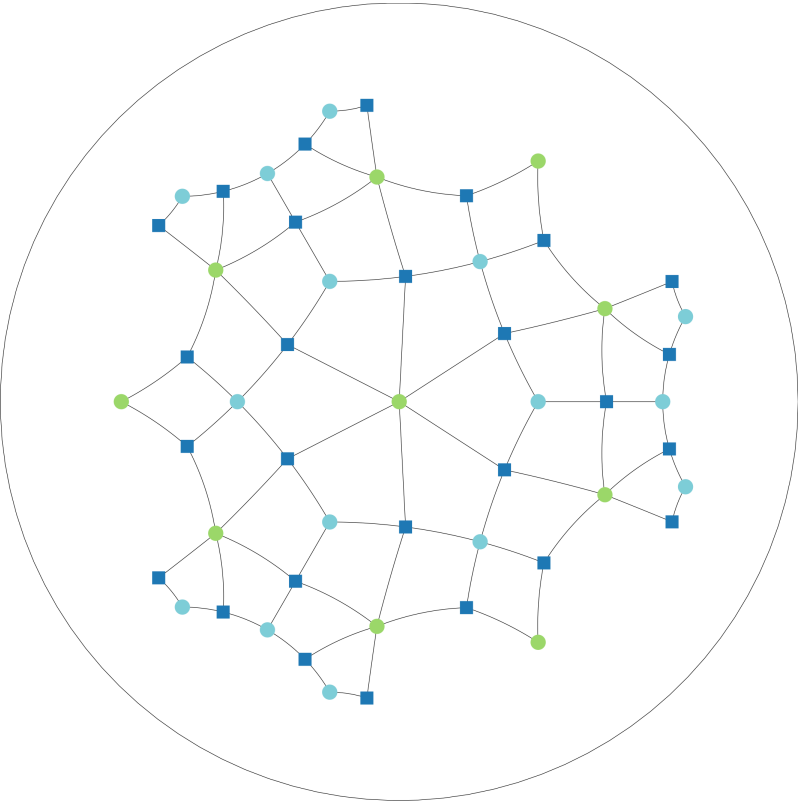
$(3, 3, 4)$ -hyperbolic surface code

Triangle tile



Layer 0 without trimming





Error-threshold summary

Code family	Estimated p_{th}	95% confidence interval
Surface code	13.35%	12.87–13.73%
Hyperbolic (2, 3, 7)	14.09%	13.96–14.22%
Hyperbolic (2, 4, 5)	15.25%	15.10–15.42%
Hyperbolic (3, 3, 4)	12.83%	12.67–12.97%

VinAl turns a lattice into a 3D hyperbolic chamber

```
from vinal import VinAl

# [-3] ⊕ A3: a 4 × 4 form of signature (3,1)
G = [[-3, 0, 0, 0],
      [ 0, 2, -1, 0],
      [ 0, -1, 2, -1],
      [ 0, 0, -1, 2]]

roots = VinAl(G).run()
```

integer Gram matrix



simple reflection roots



Coxeter chamber in H^3

VinAl has a stable **SageMath** implementation and an experimental **Python/SymPy** implementation.

Bogachev & Perepechko, [software implementation](#); [VinAl source code](#).

Vinberg's algorithm discovers the walls one by one

1. Base point

Choose a point
inside $\mathbb{H}^3$.

2. Candidate mirrors

Enumerate lattice
roots by distance.

3. Accept walls

Keep mirrors
compatible with
earlier walls.

4. Stop

Recognize a finite-
volume chamber.

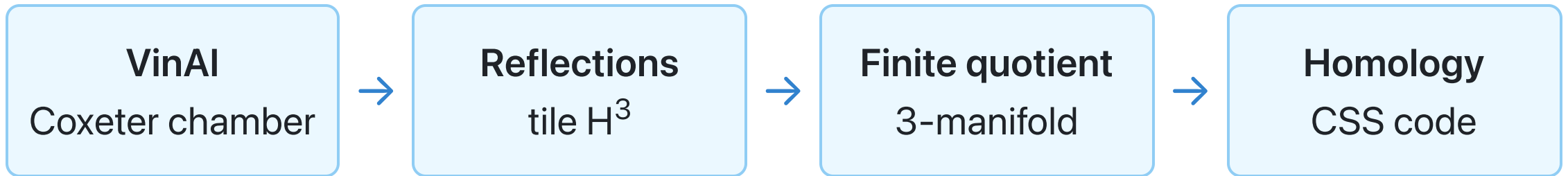
Each accepted root r gives a reflection

$$s_r(x) = x - 2 \frac{\langle x, r \rangle}{\langle r, r \rangle} r.$$

Intuition: the roots are mirror normals; the final chamber is one tile of a reflection tessellation of $\mathbb{H}^3$.

[Bogachev–Perepechko implementation](#). For a non-reflective lattice, the search need not terminate.

The chamber is only the first step toward a 3D code



Choose a torsion-free finite-index subgroup to obtain a finite cellulation M :

$$C_3(M) \longrightarrow C_2(M) \longrightarrow C_1(M) \longrightarrow C_0(M).$$

For qubits on edges, incidence matrices give the X - and Z -checks, and

$$k = \dim H_1(M; \mathbb{F}_2).$$

VinAI finds the chamber; the finite quotient and homology calculation are further work.

4D codes achieve constant block overhead

Guth and Lubotzky proved

$$k = \Theta(n), \quad d \geq n^\varepsilon \quad \implies \quad \frac{n}{k} = \Theta(1).$$

The number of data qubits per logical qubit therefore stays bounded as the code grows.

- Later bounds give $d \geq c n^{0.2}$, while this construction has $d = O(n^{0.3})$.
- An explicit family has $k/n \geq 13/72 \approx 18\%$, so $n/k \leq 5.54$ asymptotically.

This is code-block overhead only—not total fault-tolerant hardware overhead.

[Guth–Lubotzky](#); [Murillo](#); [Breuckmann–Londe](#).

Rigidity makes 3D a compelling test—not a theorem

Why 3D is promising

- Mostow rigidity starts in dimension 3.
- Topology fixes the closed hyperbolic geometry.
- Recent 3-manifold codes can have constant rate.

What must still be proved

- The 4D rate proof uses middle-dimensional homology and Euler characteristic—not rigidity.
- 3D rate, distance, and thresholds need independent estimates.

Rigidity motivates the search; homology and systoles decide the code.

Next step: enumerate Vinberg chambers and finite quotients, compute k and d , then test decoders and thresholds.

Thank you

Questions